Indexing Pipelines

the database is the truth; the index follows it

Watch the 4-min video

In one line: keep the database as the source of truth and stream its changes to the search index asynchronously, with **change data capture** into a queue and an **idempotent, versioned, bulk** indexer; rebuild indexes behind an **alias** and swap it atomically for zero-downtime reindexing.

1 The database says $79, search says $99

Products live in Postgres; search runs on Elasticsearch. A price changes but search shows the old one; a deleted product still appears. Querying is the easy part of search. The hard part is keeping a **derived copy** in step with the source of truth, through crashes, retries, deploys and schema changes. Search documents are also usually **denormalised**: one product document joins the product row, its brand name, current price, stock level and category path from several tables.

2 Why synchronous dual writes drift

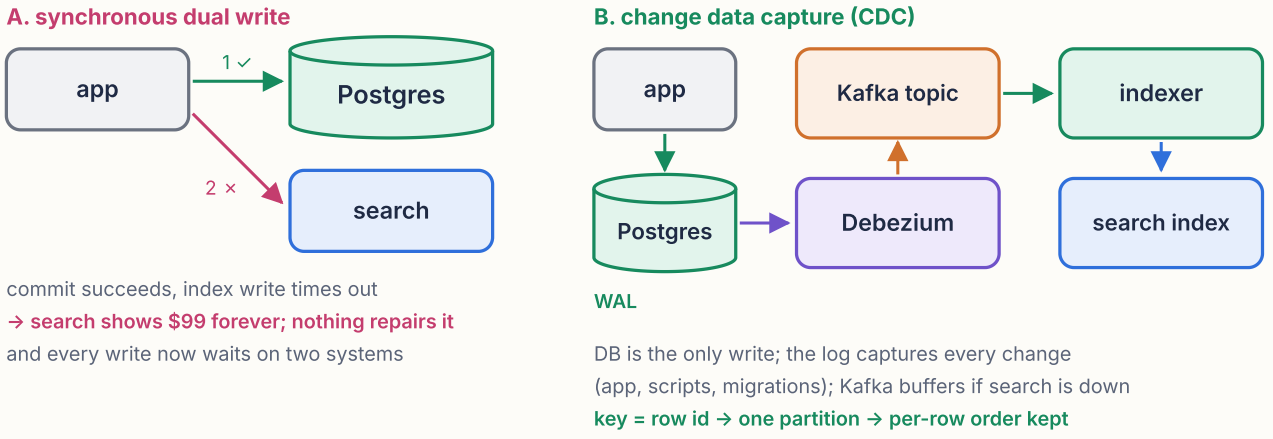


Figure 1. Dual writes have no transaction across two systems. CDC makes the database's own log the single stream of changes.

Writing to the database and then the index from the request handler looks simple, but there is no atomic commit across the two. If the index write fails after the commit (timeout, deploy, crash), they diverge with nothing to repair it. Two concurrent requests can also reach the index in the opposite order from the database. And the user's write latency and availability now depend on the search cluster too.

3 Change data capture into a queue

CDC reads the database's own replication log instead of trusting the application. **Debezium** on Postgres uses logical decoding (the `pgoutput` plugin) through a **replication slot**, turning every committed insert, update and delete into an event with before/after values and the log position (LSN). On MySQL it reads the binlog. Events go to a Kafka topic per table, keyed by primary key, so all changes to one row land in one partition, in order. Changes made by scripts, admin tools or other services are captured too, and nothing is published for a transaction that rolled back.

The alternative when you can't use CDC is the **transactional outbox**: the app writes the row and an 'event' row in the same database transaction, and a relay publishes outbox rows to the queue. Either way the database commit is the single source of events.

Operational catch: a Postgres replication slot makes the database keep WAL until the consumer confirms it. If Debezium is down for a day, WAL piles up and can fill the disk, so monitor slot lag (and on recent Postgres, cap it with `max_slot_wal_keep_size`).

4 The indexer: bulk, idempotent, versioned

- **Bulk API**: send hundreds to a few thousand documents (a few MB) per `_bulk` request, not one HTTP call per document. A bulk response can partly fail: check `errors` and per-item status, retry 429 (rejected, queue full) with backoff, and send poison documents (mapping errors) to a dead-letter topic.
- **Idempotent**: use the database primary key as `_id`, so a replayed event overwrites the same document instead of creating a duplicate. At-least-once delivery from Kafka is then safe.
- **Versioned**: index with `version_type=external` and the event's LSN or `updated_at` as the version. Elasticsearch rejects any write whose version is not higher than the stored one, so a late retry can't overwrite newer data.
- **Fan-out**: when a brand is renamed, every product document embedding it must be re-indexed; the indexer needs a lookup from brand to product IDs, or the brand should be a separate filterable field fetched at query time.
- **Backpressure**: if search is slow, the consumer lags and Kafka holds the events; alert on **consumer lag**, which is exactly your search staleness (lesson 20).



Figure 2. External versioning makes the index converge to the newest state whatever order events arrive in.

5 Rebuilding with aliases

Some changes can't be applied in place: a new analyzer, a changed field type, more primary shards (lesson 16). So the app never talks to a real index; it searches and writes an **alias** such as `products`. To rebuild:

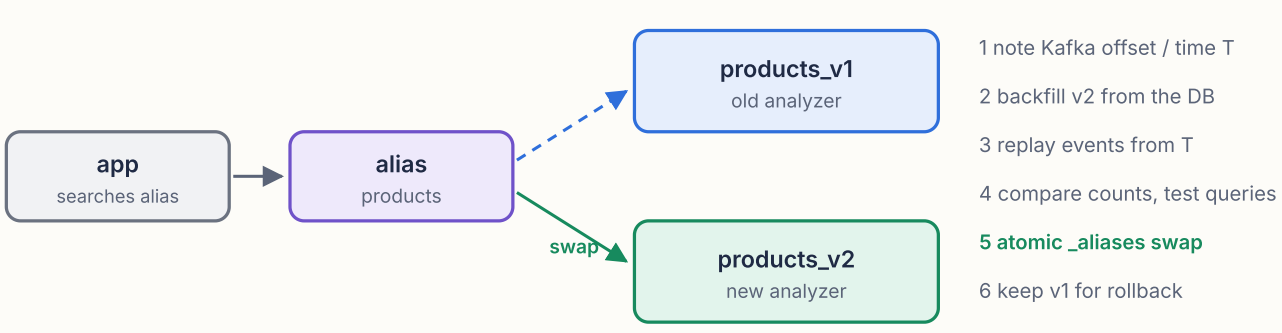


Figure 3. Zero-downtime reindex. The alias moves in one cluster-state update, so no search ever sees a half-built index.

Record the current Kafka offset (or a timestamp) *before* starting, backfill `products_v2` from a database snapshot or with the `_reindex` API, then replay events from the recorded offset. Overlap is harmless because writes are idempotent and versioned. Speed the backfill with `refresh_interval: -1` and zero replicas, then restore them. Verify counts and a set of test queries, then swap:

```
POST _aliases
{ "actions": [
  { "remove": { "index": "products_v1", "alias": "products" } },
  { "add":    { "index": "products_v2", "alias": "products", "is_write_index": true } } ] }
```

Figure 4. Both actions apply atomically. Rolling back is the same call with the indices exchanged.

WORKED EXAMPLE: POSTGRES → DEBEZIUM → KAFKA → ELASTICSEARCH

A seller changes a price from $99 to $79. Postgres commits it; Debezium reads the WAL entry and publishes `{op: "u", after: {id: 42, price: 79}, lsn: …}` to `shop.public.products`, keyed by 42. The indexer consumes a batch of 500 events, loads brand and stock for each product, and sends one `_bulk` request with `_id = 42` and `version = lsn`. After the next refresh (≤ 1 s) search shows $79; end-to-end lag is typically a second or two. A delete event becomes a bulk `delete` for `_id 42`. When the team adds a new analyzer, they build `products_v2`, catch it up from Kafka and swap the alias without users noticing.

TRADE-OFFS

- **Async pipeline**: reliable and decoupled, but search is eventually consistent (lesson 20).
- **CDC** captures everything without app changes but couples you to the DB log format and needs Kafka/Connect ops.
- **Outbox** gives explicit domain events but every writer must use it.
- **Full rebuilds** are safe with aliases but need double disk space while both indices exist.

COMMON MISTAKES

- Letting Elasticsearch auto-generate IDs, so retries create duplicate documents.
- Ignoring per-item errors in bulk responses: some documents silently never get indexed.
- Starting the backfill and only then noting the replay position, leaving a gap of lost changes.
- Pointing the application at the concrete index name instead of an alias.

6 Key terms

CDC: stream committed DB changes from its log

Transactional outbox: event row written in the same transaction

External versioning: only accept writes with a higher version

Replication slot: Postgres bookmark that retains WAL for a consumer

Bulk API: many index/delete ops in one request

Index alias: stable name that points to a real index

7 Interview questions

Q1. How do you reindex with zero downtime? BASIC

Search through an alias, build the new index in the background, catch it up from the change stream, swap the alias atomically, and keep the old index for rollback.

Q2. How do you keep the index in sync with the database? BASIC
Asynchronously, with change data capture (or an outbox) into a queue, and idempotent, versioned bulk writes keyed by the database ID.

Q3. The indexer was down for 2 hours. What happens and what do you check? SCENARIO
Kafka retained the events, so it resumes from its committed offset and catches up; check consumer lag, Postgres replication-slot WAL growth and bulk rejections while it catches up.

Q4. Search shows a product that was deleted yesterday. Where do you look? SCENARIO
Was the delete event produced (CDC), consumed (lag, DLQ), and applied (bulk item errors, version conflicts)? Then run reconciliation to find other strays (lesson 20).

Q5. Why key Kafka messages by product ID? SCENARIO
All events for one product go to one partition, so they are consumed in commit order; different products are processed in parallel.

Q6. Why isn't 'write DB, then write index' in the same request good enough? TRICKY
There is no atomic commit across them; a failure after the DB commit leaves them inconsistent with no repair path, and concurrent writes can reach the index out of order.

Q7. With external versioning, how can a deleted document come back? TRICKY
Delete tombstones are kept only for index.gc_deletes (60 s by default); an older update arriving later recreates it. Use soft deletes (a deleted flag) or ensure ordering per key.

Next: SD-P-007 · Search Consistency backendroadmap.com